# Final Project: Python Programming for Real-World Data

**Weight:** 20%

**Submission deadline to D2L:** March 30, 2026

**Presentations:** First and second weeks of April

## 1. Project Overview

In this final project, students will work with a **large, real-world dataset** and apply the Python programming skills learned throughout the course. The goal is twofold:

1. **Programming-focused objective:**
   To demonstrate good programming practices used by professional data scientists, including modular code design, use of functions and classes, clear structure, and effective use of Python libraries.
2. **Data-science objective:**
   To understand a real dataset, identify its challenges, preprocess it appropriately, perform basic analyses, and communicate findings using statistics and visualizations.

This project emphasizes **how you write and organize code**, not just the final results.

# 2. Dataset Selection

Each student (or group, if applicable) must select a **large, real-world dataset**. The dataset should be complex enough to require meaningful preprocessing.

## Dataset Requirements

- At least **10,000 rows** (or comparable size for non-tabular data)
- Real-world data (not synthetic)
- Contains **at least some challenges**, such as:
  - Missing values
  - Inconsistent entries
  - Multiple variables/features
  - Mixed data types (numerical, categorical, dates, etc.)

## Suggested Data Sources

- Kaggle
- UCI Machine Learning Repository
- Government open data portals (e.g., Canada Open Data)
- World Bank / WHO datasets
- Sports, finance, climate, health, or social datasets

The dataset **must be approved** by the instructor before proceeding.

# 3. Project Tasks

## 3.1 Data Understanding and Challenges

- Describe the dataset and its context
- Identify and explain challenges in the data:
    - Missing values
    - Outliers
    - Data type issues
    - Inconsistent or noisy entries
- Explain *why* these issues matter for analysis

## 3.2 Data Preprocessing (Using Python)

You must use **Python programming tools** covered in the course, including but not limited to:

- `list`, `dict`, loops, and conditionals
- NumPy
- Pandas
- Functions
- Classes
- Plotting libraries (Matplotlib)

Examples of preprocessing tasks:

- Handling missing values (removal, imputation, etc.)
- Data cleaning and transformation
- Feature creation
- Data type conversion
- Filtering and grouping data

**Hard-coded or copy-pasted solutions will be penalized.**
Reusable, well-structured code is expected.

## 3.3 Programming Design & Professional Practices

A major focus of this project is **how you program**, not just what you compute.

You must demonstrate:

- Use of **functions** to avoid repeated code
- Use of **classes** where appropriate (e.g., dataset handler, analyzer, visualizer)
- Clear variable naming
- Modular code organization
- Code readability and commenting
- Logical file and function structure

You should explicitly reflect on:

- How functions improved your workflow
- Why classes were useful (or why they were not)
- How your code became more readable and reusable
- What habits resemble those of professional programmers

## 3.4 Data Analysis and Visualization

Perform **basic exploratory analysis**, such as:

- Summary statistics (mean, median, std, etc.)
- Group-wise analysis
- Correlation analysis (if applicable)

Create **clear and meaningful visualizations**, such as:

- Histograms
- Line plots
- Bar charts
- Scatter plots

All figures must:

- Have titles
- Have labeled axes
- Be referenced in the report

# 4. Final Report Requirements

You must submit a **10-page written report** (excluding references and appendices).

## Report Structure

### Part I – Programming & Code Design (6 pages)

This section focuses on *programming*, not the science.

Suggested content:

- Overview of your code structure
- Explanation of key functions
- Explanation of any classes used
- Discussion of design decisions
- How Python tools (functions, classes, libraries) improved your solution
- Reflection on professional programming habits learned

Include:

- Code snippets (not full scripts)
- Explanations of *why* code was written a certain way

### Part II – Scientific/Data Report (4 pages)

This section focuses on the **data and findings**, similar to a short scientific report.

Suggested structure:

- Introduction to the dataset and topic
- Methods (brief)
- Results (statistics and plots)
- Discussion of findings
- Limitations

# 5. Submission Requirements

- **Written report** (PDF)
- **Python code files** (`.py` or `.ipynb`)
- Clear instructions on how to run the code
- Figures embedded in the report

# 6. In-Class Presentation

Each student must give an **in-class presentation (10–15 minutes)** during the **first and second weeks of April**.

The presentation should include:

- A brief overview of the dataset and research question
- Key data challenges and preprocessing steps
- Overview of the code structure (functions, classes, workflow)
- Selected results and visualizations
- Key lessons learned from a programming perspective

The presentation should focus on **programming design and workflow**, not only results.

# 7. Evaluation Criteria (High-Level)

Your project will be evaluated based on:

- Quality and structure of Python code
- Appropriate use of functions and classes
- Data preprocessing quality
- Clarity of analysis and visualization
- Quality of written explanation
- Professionalism and organization

# 8. Academic Integrity

- You may discuss ideas with classmates, but **all code and writing must be your own**
- Use of AI tools must follow course policy
- All external sources must be cited

# 9. Submission

- Zip your entire project folder.
- Use the following naming format:
  **lastname-firstname-Final-Project.zip**
- Upload the ZIP file to the **"Final Project – Python Programming for Real-World Data"** folder on D2L.

## Rubric:

| Category | Excellent (A) | Good (B) | Satisfactory (C) | Poor (D/F) | Weight |
|---|---|---|---|---|---|
| **Dataset Selection & Understanding** | Dataset is large, appropriate, well-justified; challenges are clearly identified and well explained | Dataset is appropriate; most challenges identified | Dataset is basic or small; limited discussion of challenges | Dataset inappropriate or poorly described | **5%** |
| **Data Preprocessing & Cleaning** | Comprehensive preprocessing: missing values, inconsistencies, and data types handled correctly and thoughtfully | Most preprocessing steps done correctly; minor issues | Basic preprocessing only; important issues missing | Little or incorrect preprocessing | **15%** |
| **Use of Python Tools (NumPy, Pandas, Plotting)** | Excellent, correct, and efficient use of libraries; code is idiomatic | Good use of libraries with minor inefficiencies | Basic use of libraries; limited functionality | Incorrect or minimal use of libraries | **10%** |
| **Functions & Code Modularity** | Code is well-modularized using functions; no redundancy; functions are reusable | Good use of functions; minor repetition | Some functions used but code remains repetitive | Little to no use of functions | **10%** |
| **Use of Classes & Object-Oriented Design** | Classes are well designed, appropriate, and improve code clarity and reuse | Classes used appropriately but with minor design issues | Minimal or forced use of classes | No meaningful use of classes | **10%** |
| **Code Readability & Professional Practices** | Excellent naming, structure, comments, and formatting; reflects professional coding habits | Mostly readable with minor clarity issues | Code is understandable but poorly organized | Code is hard to read, poorly named, or disorganized | **10%** |
| **Exploratory Data Analysis & Statistics** | Appropriate statistical analyses with clear interpretation | Mostly correct analyses: interpretation may be shallow | Limited or basic analysis | Incorrect or missing analysis | **10%** |
| **Visualization Quality** | Clear, informative plots with titles, labels, and correct interpretation | Plots are mostly clear; minor labeling or design issues | Basic plots with limited explanation | Poor or missing visualizations | **10%** |
| **Programming Reflection (6-page section)** | Insightful reflection on programming decisions, tools, and professional habits | Good reflection with reasonable explanations | Mostly descriptive; limited reflection | Minimal or missing reflection | **5%** |
| **Scientific Report (4-page section)** | Clear, concise, and well-structured scientific report | Mostly clear with minor issues | Basic reporting; lacks depth | Poorly written or unclear | **5%** |
| **Presentation (10-15 minutes)** | Clear, well-structured, and professional presentation; effectively explains dataset, programming design (functions/classes), preprocessing, and key results; excellent slides and delivery | Clear presentation covering most required elements; minor issues with structure, timing, or clarity | Basic presentation; limited discussion of programming design or results; slides or delivery lack clarity | Unclear, disorganized, or incomplete presentation; key elements missing | **10%** |

## Notes for Students

- This project is **code-focused**, not just results-focused.
- Clean, modular, readable code matters more than complex analysis.
- Thoughtful reflection on *how* you programmed is essential.